You:

hello

ChatGPT:

Hi there! How can I assist you today?

You:

Thiết kế cơ sở dữ liệu SQL server cho đề bài sau, biết rằng sản phẩm cụ thể ở đây là Sách Đề bài: Chương trình WPF C# quản lí việc bán hàng cho một cửa hàng nhỏ. Bạn có thể chọn một trong các chủ đề gợi ý sau: 1. Laptop 2. Điện thoại di động 3. Sách 4. Chủ đề nào mà bạn muốn Các chức năng cơ sở (5 điểm) Cơ sở dữ liệu sử dụng: SQL Server (hoặc CSDL quan hệ tùy chọn tương đương như Postgres/MySQL/MariaDB/MongoDB…) hoặc CSDL mà bạn muốn. 1. Màn hình đăng nhập (0.25 điểm) Cho nhập username và password để đi vào màn hình chính. Có chức năng lưu username và password ở local để lần sau người dùng không cần đăng nhập lại. Password cần được mã hóa. Cho phép cấu hình thêm thông tin như server, tên database kết nối. 2. Màn hình dashboard (0.25 điểm) Cung cấp tổng quan về hệ thống đang quản lí, ví dụ: • Có tổng cộng bao nhiêu sản phẩm đang bán • Có tổng cộng bao nhiêu đơn hàng mới trong tuần / tháng • Liệt kê top 5 sản phẩm đang sắp hết hàng (số lượng < 5) • Ví dụ một dashboard thường thấy sẽ như thế này 3. Quản lí hàng hóa (1.5 điểm) • Import dữ liệu gốc ban đầu (loại sản phẩm, danh sách các sản phẩm) từ tập tin Excel hoặc Access. • Thao tác với Loại sản phẩm: Xem danh sách, Thêm, Xóa, Cập nhật • Thao tác với Sản phẩm o Xem danh sách theo Loại sản phẩm □ Có phân trang □ Sắp xếp theo tiêu chí o Xem chi tiết một sản phẩm □ Xóa, cập nhật sản phẩm o Thêm mới một sản phẩm • Cho phép tìm kiếm sản phẩm theo tên • Cho phép lọc lại sản phẩm theo khoảng giá 4. Quản lí các đơn hàng (1.5 điểm) • [ ] Tạo ra các đơn hàng • [ ] Cho phép xóa một đơn hàng, cập nhật một đơn hàng • [ ] Cho phép xem danh sách các đơn hàng có phân trang, xem chi tiết một đơn hàng • [ ] Tìm kiếm các đơn hàng từ ngày đến ngày 5. Báo cáo thống kê (1 điểm) • Báo cáo doanh thu và lợi nhuận theo ngày đến ngày, theo tuần, theo tháng, theo năm (vẽ biểu đồ) • Xem các sản phẩm và số lượng bán theo ngày đến ngày, theo tuần, theo tháng, theo năm (vẽ biểu đồ) 6. Cấu hình (0.25 điểm) • Cho phép hiệu chỉnh số lượng sản phẩm mỗi trang • Cho phép khi chạy chương trình lên thì mở lại màn hình cuối mà lần trước tắt

---

ChatGPT:

Dựa trên yêu cầu của bạn, tôi sẽ thiết kế một cơ sở dữ liệu SQL Server cho chương trình WPF C# quản lý việc bán hàng với sản phẩm cụ thể là Sách. Cấu trúc cơ sở dữ liệu sẽ bao gồm các bảng chính sau:

1. **Bảng Users**: Lưu thông tin người dùng, bao gồm username, password (mã hóa), và các cấu hình khác.
2. **Bảng Products**: Lưu thông tin về sách, bao gồm ID, tên sách, giá, số lượng, và thông tin chi tiết khác.
3. **Bảng ProductCategories**: Lưu các loại sách khác nhau.
4. **Bảng Orders**: Lưu thông tin các đơn hàng, bao gồm ID đơn hàng, ngày đặt hàng, và thông tin chi tiết đơn hàng.
5. **Bảng OrderDetails**: Lưu chi tiết của mỗi đơn hàng, bao gồm ID sản phẩm, số lượng, và giá.

Dưới đây là mô tả chi tiết cho mỗi bảng:

Bảng Users

- UserId: INT, PRIMARY KEY, AUTO_INCREMENT
- Username: VARCHAR
- Password: VARCHAR (mã hóa)
- ConfigurationSettings: JSON (hoặc một dạng lưu trữ cấu hình tùy chọn)

Bảng Products

- ProductId: INT, PRIMARY KEY, AUTO_INCREMENT
- ProductName: VARCHAR
- Price: DECIMAL
- Quantity: INT
- CategoryId: INT (FOREIGN KEY to ProductCategories)
- Description: TEXT

Bảng ProductCategories

- CategoryId: INT, PRIMARY KEY, AUTO_INCREMENT
- CategoryName: VARCHAR

Bảng Orders

- OrderId: INT, PRIMARY KEY, AUTO_INCREMENT

- UserId: INT (FOREIGN KEY to Users)
- OrderDate: DATETIME
- TotalAmount: DECIMAL

Bảng OrderDetails

- OrderDetailId: INT, PRIMARY KEY, AUTO_INCREMENT
- OrderId: INT (FOREIGN KEY to Orders)
- ProductId: INT (FOREIGN KEY to Products)
- Quantity: INT
- Price: DECIMAL

Để hỗ trợ các chức năng như tìm kiếm, lọc, báo cáo, và thống kê, bạn cần xây dựng các truy vấn SQL phức tạp hơn, kết hợp với việc sử dụng các Stored Procedure và Function trong SQL Server. Bạn cũng cần thiết kế giao diện WPF để tương tác với cơ sở dữ liệu này thông qua các chức năng như thêm, sửa, xóa và truy vấn dữ liệu.

You:

Tôi cần khai báo các thuộc tính đó trong WPF cụ thể là các biến trong file .cs như thế nào

---

ChatGPT:

Để khai báo các thuộc tính trong WPF phù hợp với cấu trúc cơ sở dữ liệu đã thiết kế, bạn cần tạo các lớp (class) tương ứng trong file .cs. Dưới đây là một ví dụ về cách bạn có thể khai báo các lớp và thuộc tính cho các bảng dữ liệu như User, Product, ProductCategory, Order, và OrderDetail trong C#.

1. User Class

```
csharp
public class User
{
    public int UserId { get; set; }
    public string Username { get; set; }
    public string Password { get; set; } // Lưu ý đây là password đã được mã hóa
    public string ConfigurationSettings { get; set; } // JSON hoặc dạng string khác
}
```

2. Product Class

```
csharp
public class Product
{
    public int ProductId { get; set; }
    public string ProductName { get; set; }
    public decimal Price { get; set; }
    public int Quantity { get; set; }
    public int CategoryId { get; set; }
    public string Description { get; set; }
}
```

3. ProductCategory Class

```
csharp
public class ProductCategory
{
    public int CategoryId { get; set; }
    public string CategoryName { get; set; }
}
```

4. Order Class

```
csharp
public class Order
{
```

```
    public int OrderId { get; set; }
    public int UserId { get; set; }
    public DateTime OrderDate { get; set; }
    public decimal TotalAmount { get; set; }
}
```

5. OrderDetail Class

```
csharp
public class OrderDetail
{
    public int OrderDetailId { get; set; }
    public int OrderId { get; set; }
    public int ProductId { get; set; }
    public int Quantity { get; set; }
    public decimal Price { get; set; }
}
```

Các lớp này đại diện cho cấu trúc dữ liệu trong cơ sở dữ liệu của bạn và được sử dụng để trao đổi dữ liệu giữa ứng dụng WPF và cơ sở dữ liệu SQL Server. Bạn có thể sử dụng các lớp này để tạo ra các đối tượng tương ứng và thao tác dữ liệu (thêm mới, cập nhật, xóa, truy vấn) thông qua các kết nối và truy vấn cơ sở dữ liệu.

Hãy lưu ý rằng mật khẩu trong lớp User nên được lưu trữ dưới dạng mã hóa để đảm bảo an toàn thông tin. Bạn cũng có thể cần thêm các thuộc tính hoặc phương thức khác tùy thuộc vào nhu cầu cụ thể của ứng dụng.

You:

Thực hiện cụ thể các đoạn code cho việc "mật khẩu" được nhắc ở trên

---

ChatGPT:

Để mã hóa mật khẩu trong ứng dụng WPF C#, bạn có thể sử dụng một số phương pháp mã hóa mật khẩu phổ biến như hash bằng SHA256. Dưới đây là một ví dụ về cách thực hiện mã hóa mật khẩu:

1. Tạo Một Lớp Helper Để Mã Hóa Mật Khẩu

```
csharp
using System.Security.Cryptography;
using System.Text;

public static class PasswordHelper
{
    public static string HashPassword(string password)
    {
        using (SHA256 sha256 = SHA256.Create())
        {
            byte[] hashedBytes = sha256.ComputeHash(Encoding.UTF8.GetBytes(password));
            string hash = BitConverter.ToString(hashedBytes).Replace("-", "").ToLower();
            return hash;
        }
    }
}
```

Trong đoạn code này, phương thức `HashPassword` sẽ nhận vào một chuỗi là mật khẩu và trả về một chuỗi đã được mã hóa bằng SHA256.

2. Sử Dụng Lớp Helper Trong Quá Trình Đăng Ký Hoặc Cập Nhật Mật Khẩu

Khi người dùng đăng ký hoặc cập nhật mật khẩu, bạn sẽ gọi phương thức `HashPassword` để mã hóa mật khẩu trước khi lưu vào cơ sở dữ liệu.

```
csharp
public void RegisterUser(string username, string password)
{
    string hashedPassword = PasswordHelper.HashPassword(password);
```

```
    // Lưu username và hashedPassword vào cơ sở dữ liệu
}
```

3. Kiểm Tra Mật Khẩu Khi Đăng Nhập

Khi người dùng đăng nhập, bạn cũng sẽ mã hóa mật khẩu nhập vào và so sánh với mật khẩu đã mã hóa lưu trong cơ sở dữ liệu.

```
csharp
public bool ValidateUser(string username, string password)
{
    string hashedPassword = PasswordHelper.HashPassword(password);

    // Kiểm tra trong cơ sở dữ liệu xem có username với hashedPassword tương ứng không
    // Trả về true nếu đúng, false nếu không
}
```

Lưu ý rằng mã hóa mật khẩu là một bước quan trọng để bảo vệ thông tin cá nhân của người dùng. Tuy nhiên, bạn cũng cần đảm bảo rằng hệ thống của mình có các biện pháp bảo mật khác như HTTPS, xác thực hai yếu tố, v.v. để tăng cường bảo mật.

You:

Cho tôi biết table ProductCategory sẽ show ra màn hình điều gì

ChatGPT:

Trong ứng dụng WPF C# quản lý việc bán sách, bảng `ProductCategory` sẽ được sử dụng để hiển thị các loại sách khác nhau. Khi bạn hiển thị dữ liệu từ bảng `ProductCategory` lên màn hình, bạn có thể chọn hiển thị các thông tin sau:

1. **Category ID (CategoryId)**: Đây là mã định danh duy nhất cho mỗi loại sách. Nó thường được sử dụng trong các truy vấn dữ liệu để liên kết sách với loại sách của chúng.
2. **Category Name (CategoryName)**: Tên của loại sách. Đây là thông tin mà người dùng sẽ thấy và sử dụng để nhận biết các loại sách khác nhau, ví dụ như "Văn học", "Khoa học", "Trẻ em", v.v.

Cách thức hiển thị này có thể được thực hiện thông qua một danh sách đơn giản (ví dụ: `ListBox` hoặc `ComboBox` trong WPF) hoặc thông qua một bảng dữ liệu phức tạp hơn (như `DataGrid` trong WPF) tùy thuộc vào yêu cầu của ứng dụng.

Nếu bạn sử dụng `DataGrid` để hiển thị, người dùng không chỉ có thể xem danh sách các loại sách, mà còn có thể thực hiện các thao tác như sắp xếp, lọc, thêm mới, chỉnh sửa hoặc xóa các loại sách ngay trên giao diện.

Đây là một ví dụ đơn giản về cách bạn có thể hiển thị dữ liệu từ bảng `ProductCategory` trong một `DataGrid` trong WPF:

```
xml
<DataGrid AutoGenerateColumns="True" ItemsSource="{Binding ProductCategories}">
    <DataGrid.Columns>
        <DataGridTextColumn Header="Category ID" Binding="{Binding CategoryId}" />
        <DataGridTextColumn Header="Category Name" Binding="{Binding CategoryName}" />
    </DataGrid.Columns>
</DataGrid>
```

Trong đoạn code trên, `ItemsSource` của `DataGrid` được liên kết (`Binding`) với một danh sách các đối tượng `ProductCategory` từ code-behind hoặc ViewModel. Mỗi `ProductCategory` trong danh sách sẽ có `CategoryId` và `CategoryName` được hiển thị trong các cột tương ứng.

You:

Giải thích cho tôi hiểu rõ về mối quan hệ giữa ProductCategory và product

---

ChatGPT:

Mối quan hệ giữa `ProductCategory` và `Product` trong cơ sở dữ liệu là một ví dụ điển hình của mối quan hệ "Một-đến-Nhiều" (One-to-Many). Trong mối quan hệ này, mỗi danh mục sản phẩm (ProductCategory) có thể có nhiều sản phẩm (Products) liên kết với nó, nhưng mỗi sản phẩm chỉ thuộc về một danh mục cụ thể.

Mô tả Mối Quan Hệ:

1. **ProductCategory (Một)**: Bảng này chứa thông tin về các danh mục sản phẩm. Trong trường hợp của bạn, đó là các loại sách. Mỗi danh mục được định danh bằng một `CategoryId` duy nhất.
2. **Product (Nhiều)**: Bảng này chứa thông tin chi tiết về từng sản phẩm (sách), bao gồm tên sách, giá, số lượng, vv. Mỗi sản phẩm này có một trường (thường là `CategoryId`) để chỉ ra danh mục mà nó thuộc về.

Cách Thức Hoạt Động:

- Mỗi khi bạn thêm một sản phẩm mới vào bảng `Product`, bạn cũng cần chỉ định `CategoryId` cho nó. `CategoryId` này phản ánh danh mục mà sản phẩm thuộc về.
- Khi truy vấn dữ liệu, bạn có thể dễ dàng lấy ra tất cả các sản phẩm thuộc một danh mục cụ thể bằng cách sử dụng `CategoryId`.
- Cấu trúc này giúp tổ chức dữ liệu một cách rõ ràng và hợp lý, đồng thời hỗ trợ các thao tác tìm kiếm và lọc dữ liệu một cách hiệu quả.

Ví Dụ:

Giả sử có một danh mục có `CategoryId` là 1 và tên là "Văn Học". Trong bảng `Product`, bạn có thể có nhiều sách với `CategoryId` là 1, điều này có nghĩa là tất cả những cuốn sách này thuộc về danh mục "Văn Học".

Lợi Ích:

- **Tổ Chức Dữ Liệu**: Dễ dàng quản lý và phân loại sản phẩm.
- **Mở Rộng và Bảo Trì**: Khi cần thêm danh mục mới hoặc cập nhật thông tin danh mục, bạn chỉ cần thao tác trên bảng `ProductCategory`.
- **Truy Vấn Hiệu Quả**: Có thể lọc và tìm kiếm sản phẩm dựa trên danh mục một cách nhanh chóng.

Cách Thiết Kế Trong SQL:

Trong SQL, mối quan hệ này thường được thực hiện thông qua việc sử dụng khóa ngoại (Foreign Key). Ví dụ, `CategoryId` trong bảng `Product` sẽ là khóa ngoại liên kết với `CategoryId` trong bảng `ProductCategory`.

```
sql
CREATE TABLE ProductCategory (
    CategoryId INT PRIMARY KEY,
    CategoryName VARCHAR(255)
);

CREATE TABLE Product (
    ProductId INT PRIMARY KEY,
    ProductName VARCHAR(255),
    CategoryId INT,
    FOREIGN KEY (CategoryId) REFERENCES ProductCategory(CategoryId)
);
```

Trong mô hình này, việc thêm, cập nhật, hoặc xóa một danh mục trong `ProductCategory` có thể ảnh hưởng đến các sản phẩm liên quan trong bảng `Product`, do đó cần thực hiện các thao tác này một cách cẩn thận để đảm bảo tính toàn vẹn dữ liệu.

You:

Tham khảo 2 ảnh sau và thực hiện lại bằng phương pháp này cho việc "mật khẩu" ở trên

---

ChatGPT:

Các ảnh bạn cung cấp mô tả việc sử dụng `System.Security.Cryptography` để mã hóa và giải mã mật khẩu trong .NET. Đây là một cách tiếp cận khác so với việc chỉ sử dụng hash mật khẩu mà tôi đã mô tả trước đó. Phương pháp này cho phép bạn mã hóa mật khẩu một cách an toàn và có thể giải mã lại khi cần thiết.

Dưới đây là cách thực hiện mã hóa và giải mã mật khẩu dựa trên thông tin từ các ảnh bạn đã cung cấp:

Mã Hóa Mật Khẩu

```
csharp
using System;
using System.Security.Cryptography;
using System.Text;
using System.IO;

public static class PasswordEncryption
{
    public static string EncryptPassword(string password)
    {
        // Convert the password string to a byte array
        var passwordBytes = Encoding.UTF8.GetBytes(password);

        // Generate entropy (random data) for encryption
        var entropy = new byte[20];
        using (var rng = new RNGCryptoServiceProvider())
        {
            rng.GetBytes(entropy);
        }

        // Encrypt the password using the Windows Data Protection API
        var cipherText = ProtectedData.Protect(passwordBytes, entropy, DataProtectionScope.CurrentUser);

        // Save the entropy and the encrypted password in the configuration file
        // In a real-world application, you would save these values in a secure configuration store
        // Here, we're converting them to Base64 strings to save as plain text (for example purposes only)
        var configPassword = Convert.ToBase64String(cipherText);
        var configEntropy = Convert.ToBase64String(entropy);

        // Save or update the configuration settings as necessary
        // For example purposes, we are returning a Tuple containing the encrypted password and entropy
        return Tuple.Create(configPassword, configEntropy);
    }
}
```

Giải Mã Mật Khẩu

```
csharp
public static class PasswordDecryption
{
    public static string DecryptPassword(string encryptedPassword, string storedEntropy)
    {
        // Convert the encrypted password and entropy from Base64 strings to byte arrays
        byte[] cipherText = Convert.FromBase64String(encryptedPassword);
        byte[] entropy = Convert.FromBase64String(storedEntropy);

        // Decrypt the password
        byte[] passwordBytes = ProtectedData.Unprotect(cipherText, entropy, DataProtectionScope.CurrentUser);

        // Convert the byte array back to a string
        return Encoding.UTF8.GetString(passwordBytes);
    }
}
```

Trong đoạn code mã hóa, bạn sử dụng `ProtectedData.Protect` để mã hóa mật khẩu và `ProtectedData.Unprotect` để giải mã. `entropy` được sử dụng để tăng cường bảo mật cho quá trình mã hóa. Đảm bảo rằng `entropy` và dữ liệu mã hóa được lưu trữ một cách an toàn và chỉ có thể truy cập được bởi ứng dụng hoặc người dùng được ủy quyền.

Lưu ý: Việc sử dụng `DataProtectionScope.CurrentUser` chỉ cho phép người dùng hiện tại giải mã dữ liệu. Nếu bạn cần giải mã thông qua các người dùng khác hoặc máy khác, bạn sẽ cần sử dụng `DataProtectionScope.LocalMachine`, và lưu trữ `entropy` cẩn thận để đảm bảo an toàn dữ liệu.

You:

Lúc này tại cơ sở dữ liệu SQL server sẽ được thiết kế như thế nào

---

ChatGPT:

Khi sử dụng phương pháp mã hóa và giải mã mật khẩu như được mô tả từ các ảnh bạn đã cung cấp, cấu trúc của cơ sở dữ liệu SQL Server cần được thiết kế để lưu trữ thông tin sau:

1. **Mật khẩu đã mã hóa**: Đây là chuỗi mật khẩu sau khi đã được mã hóa. Nó sẽ được lưu trữ dưới dạng chuỗi Base64 hoặc dưới dạng binary tùy thuộc vào cách bạn chọn lưu trữ.
2. **Entropy**: Dữ liệu ngẫu nhiên (entropy) được sử dụng trong quá trình mã hóa. Mỗi mật khẩu sẽ có một giá trị entropy riêng và cần được lưu trữ cùng với mật khẩu để sử dụng trong quá trình giải mã.

Dưới đây là một ví dụ về cách thiết kế bảng `Users` trong cơ sở dữ liệu để lưu trữ thông tin người dùng, bao gồm mật khẩu và entropy:

```
sql
CREATE TABLE Users (
    UserId INT PRIMARY KEY IDENTITY,
    Username NVARCHAR(255) NOT NULL UNIQUE,
    Password VARBINARY(MAX) NOT NULL, -- Mật khẩu đã mã hóa
    Entropy VARBINARY(20) NOT NULL, -- Entropy sử dụng cho quá trình mã hóa
    -- Các trường thông tin khác nếu cần
);
```

Trong đoạn code trên, `Password` được lưu trữ dưới dạng `VARBINARY(MAX)` để có thể lưu trữ mật khẩu dưới dạng binary sau khi đã mã hóa. Tùy thuộc vào yêu cầu cụ thể của bạn, bạn cũng có thể chọn lưu trữ nó dưới dạng `NVARCHAR` nếu lưu dưới dạng chuỗi Base64.

`Entropy` cũng được lưu trữ dưới dạng `VARBINARY` để đảm bảo có thể sử dụng lại chính xác trong quá trình giải mã.

Lưu ý quan trọng: Trong thực tế, việc lưu trữ entropy cùng với mật khẩu mã hóa không phải lúc nào cũng là một phương pháp tốt nhất vì nó có thể giảm bớt lợi ích của việc sử dụng entropy. Phương pháp này chỉ nên được sử dụng khi bạn cần giải mã mật khẩu, ví dụ như để sử dụng mật khẩu cho một kết nối đã mã hóa, và bạn cần đảm bảo an ninh cơ sở dữ liệu của mình. Trong hầu hết các ứng dụng, mật khẩu nên được hash chứ không phải được mã hóa, vì hash là một chiều và không thể giải mã, tăng cường bảo mật cho thông tin đăng nhập của người dùng.

You:

Chỉ mã hóa mà không cần giải mã

ChatGPT:

Nếu mục tiêu là chỉ mã hóa mật khẩu mà không cần giải mã, thì việc sử dụng `hashing` là phương pháp thích hợp. `Hashing` là một cách một chiều để chuyển đổi mật khẩu - một khi mật khẩu đã được hash, bạn không thể chuyển đổi nó trở lại dạng ban đầu. Điều này làm tăng tính bảo mật vì ngay cả khi cơ sở dữ liệu bị xâm nhập, kẻ tấn công cũng không thể biết được mật khẩu gốc.

Trong SQL Server, bạn sẽ lưu trữ kết quả của việc hash mật khẩu, không phải mật khẩu gốc. Dưới đây là cách thiết kế bảng `Users` để lưu trữ mật khẩu đã hash:

```
sql
CREATE TABLE Users (
    UserId INT PRIMARY KEY IDENTITY,
    Username NVARCHAR(255) NOT NULL UNIQUE,
    PasswordHash VARBINARY(64) NOT NULL, -- Lưu trữ hash của mật khẩu
    -- Các trường thông tin khác nếu cần
);
```

Trong bảng này:

- `PasswordHash` sẽ lưu trữ giá trị hash của mật khẩu. Kích thước của trường này phụ thuộc vào thuật toán hash bạn chọn; ví dụ, SHA256 sẽ tạo ra một hash 32 byte (256 bit).

Khi người dùng tạo tài khoản hoặc thay đổi mật khẩu, bạn sẽ tạo hash của mật khẩu và lưu trữ hash này vào cơ sở dữ liệu:

```
csharp
using System.Security.Cryptography;
using System.Text;
```

```
public static string HashPassword(string password)
{
    using (SHA256 sha256 = SHA256.Create())
    {
        // Convert the password string to a byte array and compute the hash
        byte[] hashedBytes = sha256.ComputeHash(Encoding.UTF8.GetBytes(password));
        // Return the hashed password as a Base64 string to store in the database
        return Convert.ToBase64String(hashedBytes);
    }
}
```

Khi người dùng đăng nhập, bạn sẽ hash mật khẩu mà họ nhập vào và so sánh hash này với hash lưu trữ trong cơ sở dữ liệu:

```
csharp
public bool ValidateUser(string username, string password)
{
    // Hash the password the user entered
    string hashedPassword = HashPassword(password);

    // Retrieve the stored password hash from the database for this username
    string storedPasswordHash = // Code to retrieve from database

    // Compare the hashed passwords
    return hashedPassword == storedPasswordHash;
}
```

Lưu ý rằng bạn không bao giờ lưu trữ hoặc truyền mật khẩu gốc; chỉ hash của nó mới được lưu trữ và sử dụng để xác thực người dùng.

You:

Bây giờ hãy thiết kế đầy đủ từ màn hình .xaml, .cs, SQL server cho việc nhập mật khẩu (cụ thể là tính năng đăng nhập kiểm tra mật khẩu có tồn tại trong SQL server), mã hóa,...

ChatGPT:

Thiết kế một tính năng đăng nhập đơn giản cho một ứng dụng WPF với C# và SQL Server có thể được chia thành các bước sau:

Bước 1: Thiết kế Giao diện Đăng Nhập trong WPF (.xaml)

Tạo một file `LoginWindow.xaml` cho giao diện đăng nhập:

```
xml
<Window x:Class="YourApp.LoginWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Login" Height="200" Width="300">
    <Grid>
        <StackPanel HorizontalAlignment="Center" VerticalAlignment="Center">
            <TextBox x:Name="txtUsername" Width="200" Margin="10" PlaceholderText="Username"/>
            <PasswordBox x:Name="txtPassword" Width="200" Margin="10" PlaceholderText="Password"/>
            <Button Content="Login" Width="200" Margin="10" Click="Login_Click"/>
        </StackPanel>
    </Grid>
</Window>
```

Bước 2: Xử lý Sự kiện Đăng Nhập trong Code-behind (.cs)

Tạo file `LoginWindow.xaml.cs` để xử lý sự kiện:

```
csharp
using System;
using System.Data.SqlClient;
using System.Security.Cryptography;
using System.Text;
using System.Windows;
```

```
public partial class LoginWindow : Window
{
    public LoginWindow()
    {
        InitializeComponent();
    }

    private void Login_Click(object sender, RoutedEventArgs e)
    {
        string username = txtUsername.Text;
        string password = txtPassword.Password;

        if (ValidateUser(username, password))
        {
            // Login was successful
            MessageBox.Show("Login successful!");
            // Here you would typically open the main window of your application
        }
        else
        {
            // Login failed
            MessageBox.Show("Login failed!");
        }
    }

    private bool ValidateUser(string username, string password)
    {
        string connectionString = "Your Connection String Here";
        string hashedPassword = HashPassword(password);

        using (SqlConnection conn = new SqlConnection(connectionString))
        {
            conn.Open();
            string query = "SELECT PasswordHash FROM Users WHERE Username = @Username";
            using (SqlCommand cmd = new SqlCommand(query, conn))
            {
                cmd.Parameters.AddWithValue("@Username", username);
                var storedPasswordHash = cmd.ExecuteScalar() as string;

                if (storedPasswordHash != null && hashedPassword == storedPasswordHash)
                {
                    return true; // User is validated
                }
                else
                {
                    return false; // User validation failed
                }
            }
        }
    }

    private static string HashPassword(string password)
    {
        using (SHA256 sha256 = SHA256.Create())
        {
            byte[] hashedBytes = sha256.ComputeHash(Encoding.UTF8.GetBytes(password));
            return Convert.ToBase64String(hashedBytes);
        }
    }
}
```

Bước 3: Thiết Kế Cơ Sở Dữ Liệu SQL Server

Tạo bảng `Users` trong cơ sở dữ liệu SQL Server của bạn:

```
sql
CREATE TABLE Users (
    UserId INT PRIMARY KEY IDENTITY,
    Username NVARCHAR(255) NOT NULL UNIQUE,
    PasswordHash NVARCHAR(255) NOT NULL -- Giả sử bạn đang lưu trữ hash dưới dạng Base64
);
```

Bước 4: Tạo Kết Nối SQL Server

Trong ứng dụng của bạn, bạn sẽ cần một chuỗi kết nối SQL Server. Bạn có thể lưu trữ nó trong file `App.config` hoặc `Web.config` (tuỳ thuộc vào dạng ứng dụng của bạn), hoặc trực tiếp trong code. Đây là một ví dụ về cách đặt chuỗi kết nối:

```
xml
<connectionStrings>
    <add name="DefaultConnection"
         connectionString="Data Source=YourServerName;Initial Catalog=YourDatabaseName;Integrated Security=True"
         providerName="System.Data.SqlClient" />
</connectionStrings>
```

Và sử dụng nó trong code C# như sau:

```
csharp
string connectionString = ConfigurationManager.ConnectionStrings["DefaultConnection"].ConnectionString;
```

Để hoàn thiện, bạn cần thêm logic để xử lý trường hợp đăng nhập thành công và thất bại, và có thể thêm các tính năng bảo mật như giới hạn số lần đăng nhập thất bại, và sử dụng HTTPS để bảo vệ thông tin khi truyền qua mạng.

Lưu ý: Mã nguồn trên chỉ mang tính chất minh họa, trong môi trường thực tế cần cân nhắc thêm các vấn đề về bảo mật như SQL Injection, mã hóa kết nối, và xử lý ngoại lệ.

You:

Từ đoạn phân tích cơ sở dữ liệu SQL trên. Hãy viết tất cả ra thành đoạn mã SQL server cần cho những điều trên

---

ChatGPT:

Dựa trên phân tích trên, để thiết lập cơ sở dữ liệu SQL Server cho tính năng đăng nhập, bạn sẽ cần tạo ra bảng `Users` để lưu trữ thông tin người dùng và mật khẩu đã hash. Dưới đây là đoạn mã SQL Server đầy đủ để tạo bảng `Users` và cài đặt một stored procedure để kiểm tra mật khẩu người dùng khi đăng nhập:

Tạo Bảng Users

```
sql
IF NOT EXISTS (SELECT * FROM sys.objects WHERE object_id = OBJECT_ID(N'[dbo].[Users]') AND type in (N'U'))
BEGIN
CREATE TABLE [dbo].[Users] (
    [UserId] INT IDENTITY(1,1) NOT NULL PRIMARY KEY, -- Tự động tăng cho mỗi người dùng mới
    [Username] NVARCHAR(255) NOT NULL UNIQUE, -- Tên đăng nhập duy nhất
    [PasswordHash] NVARCHAR(255) NOT NULL -- Mật khẩu đã được hash
    -- Bạn có thể thêm các trường khác nếu cần
);
END;
GO
```

Stored Procedure Kiểm Tra Đăng Nhập

```
sql
IF NOT EXISTS (SELECT * FROM sys.objects WHERE object_id = OBJECT_ID(N'[dbo].[ValidateUser]') AND type in (N'P', N'PC'))
BEGIN
EXEC('CREATE PROCEDURE [dbo].[ValidateUser] AS BEGIN SET NOCOUNT ON; END');
END;
GO

ALTER PROCEDURE [dbo].[ValidateUser]
    @Username NVARCHAR(255),
    @PasswordHash NVARCHAR(255),
    @IsValid BIT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
```

```
    -- Kiểm tra xem có người dùng nào có tên đăng nhập và mật khẩu đã hash tương ứng không
    IF EXISTS (SELECT 1 FROM [dbo].[Users] WHERE [Username] = @Username AND [PasswordHash] = @PasswordHash)
    BEGIN
        -- Nếu có, trả về true
        SET @IsValid = 1;
    END
    ELSE
    BEGIN
        -- Nếu không, trả về false
        SET @IsValid = 0;
    END
END;
GO
```

Cách Sử Dụng Stored Procedure

Khi bạn cần xác thực người dùng từ ứng dụng C# của mình, bạn sẽ gọi Stored Procedure này như sau:

```
csharp
public bool ValidateUser(string username, string hashedPassword)
{
    bool isValid = false;
    string connectionString = // Your connection string here;

    using (SqlConnection conn = new SqlConnection(connectionString))
    {
        using (SqlCommand cmd = new SqlCommand("ValidateUser", conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            cmd.Parameters.AddWithValue("@Username", username);
            cmd.Parameters.AddWithValue("@PasswordHash", hashedPassword);
            cmd.Parameters.Add("@IsValid", SqlDbType.Bit).Direction = ParameterDirection.Output;

            conn.Open();
            cmd.ExecuteNonQuery();

            isValid = Convert.ToBoolean(cmd.Parameters["@IsValid"].Value);
        }
    }

    return isValid;
}
```

Trong đoạn code C# này, bạn sẽ hash mật khẩu người dùng nhập vào và sau đó gọi Stored Procedure với tên đăng nhập và mật khẩu đã hash để kiểm tra xem người dùng có hợp lệ không. Điều này đảm bảo rằng mật khẩu thô không bao giờ được gửi hoặc lưu trữ trong cơ sở dữ liệu, tăng cường bảo mật cho hệ thống của bạn.

You:

Ý của tôi là từ đoạn này "Dựa trên yêu cầu của bạn, tôi sẽ thiết kế một cơ sở dữ liệu SQL Server cho chương trình WPF C# quản lý việc bán hàng với sản phẩm cụ thể là Sách. Cấu trúc cơ sở dữ liệu sẽ bao gồm các bảng chính sau:"

ChatGPT:

Từ phần mô tả của bạn về ứng dụng quản lý việc bán sách, cơ sở dữ liệu SQL Server cần được thiết kế với các bảng chính sau đây:

1. Bảng `Users` để quản lý thông tin đăng nhập của người dùng.
2. Bảng `Products` để quản lý thông tin các sách.
3. Bảng `ProductCategories` để quản lý các loại sách.
4. Bảng `Orders` để quản lý thông tin đơn hàng.
5. Bảng `OrderDetails` để quản lý chi tiết đơn hàng.

Dưới đây là đoạn mã SQL Server để tạo ra cấu trúc cơ sở dữ liệu này:

```
sql
```

```
-- Tạo bảng ProductCategories
CREATE TABLE ProductCategories (
    CategoryId INT PRIMARY KEY IDENTITY,
    CategoryName NVARCHAR(255) NOT NULL
);

-- Tạo bảng Products
CREATE TABLE Products (
    ProductId INT PRIMARY KEY IDENTITY,
    ProductName NVARCHAR(255) NOT NULL,
    Price DECIMAL(10, 2) NOT NULL,
    Quantity INT NOT NULL,
    CategoryId INT NOT NULL,
    Description TEXT,
    FOREIGN KEY (CategoryId) REFERENCES ProductCategories(CategoryId)
);

-- Tạo bảng Users
CREATE TABLE Users (
    UserId INT PRIMARY KEY IDENTITY,
    Username NVARCHAR(255) NOT NULL UNIQUE,
    PasswordHash NVARCHAR(255) NOT NULL
    -- Thêm các cột cần thiết khác như thông tin cá nhân, địa chỉ, v.v.
);

-- Tạo bảng Orders
CREATE TABLE Orders (
    OrderId INT PRIMARY KEY IDENTITY,
    UserId INT NOT NULL,
    OrderDate DATETIME NOT NULL DEFAULT GETDATE(),
    TotalAmount DECIMAL(10, 2) NOT NULL,
    FOREIGN KEY (UserId) REFERENCES Users(UserId)
);

-- Tạo bảng OrderDetails
CREATE TABLE OrderDetails (
    OrderDetailId INT PRIMARY KEY IDENTITY,
    OrderId INT NOT NULL,
    ProductId INT NOT NULL,
    Quantity INT NOT NULL,
    Price DECIMAL(10, 2) NOT NULL,
    FOREIGN KEY (OrderId) REFERENCES Orders(OrderId),
    FOREIGN KEY (ProductId) REFERENCES Products(ProductId)
);
```

Đoạn mã trên tạo ra các bảng và thiết lập các khóa ngoại để định nghĩa mối quan hệ giữa các bảng. Bạn có thể thêm các ràng buộc và chỉ mục tối ưu cho việc truy vấn nếu cần.

You:

Bổ sung vào bảng Users các thuộc tính Fullname, Sex, Phone, Email. Và viết lại đoạn mã SQL

ChatGPT:

Dưới đây là đoạn mã SQL được bổ sung để tạo bảng `Users` với các thuộc tính `Fullname`, `Sex`, `Phone`, và `Email`, cùng với các bảng khác cho hệ thống quản lý bán sách:

```
sql
-- Tạo bảng ProductCategories
CREATE TABLE ProductCategories (
    CategoryId INT PRIMARY KEY IDENTITY,
    CategoryName NVARCHAR(255) NOT NULL
);

-- Tạo bảng Products
CREATE TABLE Products (
    ProductId INT PRIMARY KEY IDENTITY,
```

```
    ProductName NVARCHAR(255) NOT NULL,
    Price DECIMAL(10, 2) NOT NULL,
    Quantity INT NOT NULL,
    CategoryId INT NOT NULL,
    Description TEXT,
    FOREIGN KEY (CategoryId) REFERENCES ProductCategories(CategoryId)
);

-- Tạo bảng Users với các thuộc tính mới
CREATE TABLE Users (
    UserId INT PRIMARY KEY IDENTITY,
    Username NVARCHAR(255) NOT NULL UNIQUE,
    PasswordHash NVARCHAR(255) NOT NULL,
    Fullname NVARCHAR(255) NOT NULL,
    Sex NVARCHAR(10) NULL,
    Phone NVARCHAR(15) NULL,
    Email NVARCHAR(255) NULL
    -- Bạn có thể thêm các ràng buộc như CHECK cho Sex, và UNIQUE cho Email nếu cần
);

-- Tạo bảng Orders
CREATE TABLE Orders (
    OrderId INT PRIMARY KEY IDENTITY,
    UserId INT NOT NULL,
    OrderDate DATETIME NOT NULL DEFAULT GETDATE(),
    TotalAmount DECIMAL(10, 2) NOT NULL,
    FOREIGN KEY (UserId) REFERENCES Users(UserId)
);

-- Tạo bảng OrderDetails
CREATE TABLE OrderDetails (
    OrderDetailId INT PRIMARY KEY IDENTITY,
    OrderId INT NOT NULL,
    ProductId INT NOT NULL,
    Quantity INT NOT NULL,
    Price DECIMAL(10, 2) NOT NULL,
    FOREIGN KEY (OrderId) REFERENCES Orders(OrderId),
    FOREIGN KEY (ProductId) REFERENCES Products(ProductId)
);
```

Lưu ý:

- Cột `Sex` có thể chứa giới tính của người dùng và được thiết lập là có thể NULL nếu thông tin này không bắt buộc. Bạn có thể thêm ràng buộc `CHECK` để đảm bảo chỉ chấp nhận các giá trị hợp lệ (ví dụ, 'Male', 'Female', 'Other').
- Cột `Phone` và `Email` cũng có thể chứa thông tin liên lạc của người dùng và được thiết kế để có thể NULL nếu thông tin này không bắt buộc.
- Bạn có thể muốn đảm bảo rằng email là duy nhất nếu hệ thống của bạn yêu cầu địa chỉ email duy nhất cho mỗi tài khoản. Điều này có thể được thực hiện bằng cách thêm một ràng buộc `UNIQUE` cho cột `Email`.

You:

Insert Into vào mỗi bảng ít nhất 3 dữ liệu mẫu cho tôi

ChatGPT:

Dưới đây là

You:

Bạn vẫn tiếp tục được chứ

ChatGPT:

Dưới đây là các lệnh `INSERT INTO` để thêm dữ liệu mẫu vào mỗi bảng:

Thêm Dữ Liệu vào Bảng `ProductCategories`

```
sql
INSERT INTO ProductCategories (CategoryName)
VALUES ('Văn học cổ điển'), ('Kinh tế'), ('Khoa học giả tưởng');
```

Thêm Dữ Liệu vào Bảng `Products`

Đảm bảo rằng các `CategoryId` phải tồn tại trong bảng `ProductCategories`.

```
sql
INSERT INTO Products (ProductName, Price, Quantity, CategoryId, Description)
VALUES
('Sách Văn học 1', 120000, 50, 1, 'Mô tả về sách Văn học 1'),
('Sách Kinh tế 1', 150000, 35, 2, 'Mô tả về sách Kinh tế 1'),
('Sách Khoa học 1', 200000, 20, 3, 'Mô tả về sách Khoa học 1');
```

Thêm Dữ Liệu vào Bảng `Users`

Mật khẩu ở đây sẽ là một chuỗi giả định đã được hash.

```
sql
INSERT INTO Users (Username, PasswordHash, Fullname, Sex, Phone, Email)
VALUES
('user1', 'hashed_password1', 'Nguyễn Văn A', 'Nam', '0123456789', 'user1@example.com'),
('user2', 'hashed_password2', 'Trần Thị B', 'Nữ', '0987654321', 'user2@example.com'),
('user3', 'hashed_password3', 'Lê Văn C', 'Nam', '1234567890', 'user3@example.com');
```

Thêm Dữ Liệu vào Bảng `Orders`

```
sql
INSERT INTO Orders (UserId, OrderDate, TotalAmount)
VALUES
(1, GETDATE(), 360000),
(2, GETDATE(), 300000),
(3, GETDATE(), 400000);
```

Thêm Dữ Liệu vào Bảng `OrderDetails`

Đảm bảo rằng các `OrderId` và `ProductId` phải tồn tại trong bảng `Orders` và `Products`.

```
sql
INSERT INTO OrderDetails (OrderId, ProductId, Quantity, Price)
VALUES
(1, 1, 1, 120000),
(2, 2, 2, 150000),
(3, 3, 1, 200000);
```

Đây chỉ là dữ liệu mẫu và bạn cần thay thế các giá trị như `hashed_password1` với mật khẩu thực tế đã được hash. Và bạn cũng cần đảm bảo rằng các giá trị cho `CategoryId`, `UserId`, `OrderId`, và `ProductId` phải phản ánh đúng các ID đã được tạo trong cơ sở dữ liệu của bạn.

You:

Ở đề bài phía trên tôi có nói. Đây là 1 ứng dụng quản lý sách, sẽ có liên quan nhiều đến việc sản phẩm có số lượng bao nhiêu. Việc bạn thiết kế cơ sở dữ liệu trên là chưa đầy đủ

ChatGPT:

Tôi xin lỗi về sự nhầm lẫn. Để hoàn thiện cơ sở dữ liệu cho ứng dụng quản lý sách, ngoài việc bổ sung thông tin cho bảng `Users`, chúng ta cần chắc chắn rằng cơ sở dữ liệu có thể theo dõi số lượng sách hiện có. Bảng `Products` có thể đã có một cột `Quantity` để thể hiện điều này. Bạn cũng có thể muốn thêm một bảng `Inventory` để quản lý các thay đổi về số lượng sách, như nhập hàng mới hoặc cập nhật số lượng sau mỗi đơn hàng bán ra. Tuy nhiên, dựa trên thông tin bạn đã cung cấp, chúng ta có thể giả định rằng cột `Quantity` trong bảng `Products` là đủ cho nhu cầu cơ bản.

Dưới đây là đoạn mã SQL để tạo lại các bảng với các thông tin cần thiết, bao gồm cả cập nhật cho bảng `Users` và xác nhận rằng bảng `Products` đã có cột `Quantity`:

```
sql
-- Tạo bảng ProductCategories
CREATE TABLE ProductCategories (
    CategoryId INT PRIMARY KEY IDENTITY,
    CategoryName NVARCHAR(255) NOT NULL
);

-- Tạo bảng Products với cột Quantity để theo dõi số lượng sách
CREATE TABLE Products (
    ProductId INT PRIMARY KEY IDENTITY,
    ProductName NVARCHAR(255) NOT NULL,
    Price DECIMAL(10, 2) NOT NULL,
    Quantity INT NOT NULL CHECK (Quantity >= 0), -- Số lượng sách không thể âm
    CategoryId INT NOT NULL,
    Description TEXT,
    FOREIGN KEY (CategoryId) REFERENCES ProductCategories(CategoryId)
);

-- Cập nhật bảng Users với các thuộc tính mới
CREATE TABLE Users (
    UserId INT PRIMARY KEY IDENTITY,
    Username NVARCHAR(255) NOT NULL UNIQUE,
    PasswordHash NVARCHAR(255) NOT NULL,
    Fullname NVARCHAR(255) NOT NULL,
    Sex NVARCHAR(10) NULL CHECK (Sex IN ('Male', 'Female', 'Other')), -- Ràng buộc giới tính
    Phone NVARCHAR(15) NULL,
    Email NVARCHAR(255) NULL UNIQUE -- Đảm bảo rằng email là duy nhất
);

-- Tạo bảng Orders
CREATE TABLE Orders (
    OrderId INT PRIMARY KEY IDENTITY,
    UserId INT NOT NULL,
    OrderDate DATETIME NOT NULL DEFAULT GETDATE(),
    TotalAmount DECIMAL(10, 2) NOT NULL,
    FOREIGN KEY (UserId) REFERENCES Users(UserId)
);

-- Tạo bảng OrderDetails
CREATE TABLE OrderDetails (
    OrderDetailId INT PRIMARY KEY IDENTITY,
    OrderId INT NOT NULL,
    ProductId INT NOT NULL,
    Quantity INT NOT NULL CHECK (Quantity > 0), -- Số lượng sách trong mỗi đơn hàng phải lớn hơn 0
    Price DECIMAL(10, 2) NOT NULL,
    FOREIGN KEY (OrderId) REFERENCES Orders(OrderId),
    FOREIGN KEY (ProductId) REFERENCES Products(ProductId)
);
```

Với cấu trúc này, bảng `Products` sẽ theo dõi số lượng sách hiện có. Khi một đơn hàng được thực hiện, số lượng sách trong bảng `Products` sẽ cần được cập nhật để phản ánh số lượng đã bán.

Để quản lý việc này, bạn có thể cần một stored procedure hoặc trigger để tự động cập nhật số lượng sách trong bảng `Products` mỗi khi có đơn hàng mới hoặc cập nhật đơn hàng.

You:

Khi nhân viên lập 1 đơn hàng và thực hiện mua 1 số cuốn sách, việc cập nhật lại số lượng sách như thế nào

---

ChatGPT:

Khi nhân viên lập một đơn hàng và thực hiện mua một số cuốn sách, việc cập nhật lại số lượng sách trong cơ sở dữ liệu có thể được thực hiện thông qua việc sử dụng một Stored Procedure trong SQL Server. Stored Procedure này sẽ giảm số lượng sách trong bảng `Products` dựa trên số lượng mua trong mỗi mục của đơn hàng. Dưới đây là một ví dụ về cách tạo Stored Procedure này:

Tạo Stored Procedure

```
sql
CREATE PROCEDURE UpdateProductQuantityAfterSale
    @ProductId INT,
    @SoldQuantity INT
AS
BEGIN
    -- Kiểm tra số lượng sách còn lại trước khi cập nhật
    DECLARE @CurrentQuantity INT;
    SELECT @CurrentQuantity = Quantity FROM Products WHERE ProductId = @ProductId;

    IF @CurrentQuantity >= @SoldQuantity
    BEGIN
        -- Cập nhật số lượng sách
        UPDATE Products
        SET Quantity = Quantity - @SoldQuantity
        WHERE ProductId = @ProductId;
    END
    ELSE
    BEGIN
        -- Thông báo lỗi nếu số lượng sách không đủ
        THROW 50000, 'Số lượng sách không đủ để bán', 1;
    END
END;
GO
```

Sử Dụng Stored Procedure Khi Tạo Đơn Hàng

Khi tạo một đơn hàng, bạn sẽ gọi Stored Procedure này cho mỗi sản phẩm trong đơn hàng để cập nhật số lượng sách còn lại. Đoạn mã C# sau có thể được sử dụng để gọi Stored Procedure này:

```
csharp
public void ProcessSale(int productId, int soldQuantity)
{
    string connectionString = // Your connection string here;

    using (SqlConnection conn = new SqlConnection(connectionString))
    {
        using (SqlCommand cmd = new SqlCommand("UpdateProductQuantityAfterSale", conn))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            cmd.Parameters.AddWithValue("@ProductId", productId);
            cmd.Parameters.AddWithValue("@SoldQuantity", soldQuantity);

            conn.Open();
            cmd.ExecuteNonQuery();
        }
    }
}
```

Khi một đơn hàng được lập, bạn sẽ cần gọi hàm `ProcessSale` này cho mỗi sản phẩm trong đơn hàng với số lượng tương ứng. Hàm này sẽ giảm số lượng sách còn lại trong bảng `Products` tương ứng với số lượng đã bán.

Lưu ý rằng việc xử lý ngoại lệ cần được thực hiện cẩn thận để đảm bảo tính toàn vẹn dữ liệu, đặc biệt là trong trường hợp số lượng sách còn lại không đủ để đáp ứng yêu cầu của đơn hàng.

You:

Tôi muốn lưu lại xem số lượng sách nào sách nào được bán ra, phải thiết kế bổ sung như thế nào

---

ChatGPT:

Để lưu lại thông tin về số lượng sách nào được bán ra, bạn có thể thêm một bảng `SalesRecords` vào cơ sở dữ liệu. Bảng này sẽ ghi lại mỗi giao dịch bán sách, bao gồm ID của sản phẩm, số lượng bán ra, ngày bán, và thông tin liên quan khác. Dưới đây là cấu trúc đề xuất cho bảng `SalesRecords`:

Tạo Bảng SalesRecords

```
sql
CREATE TABLE SalesRecords (
    SaleRecordId INT PRIMARY KEY IDENTITY,
    ProductId INT NOT NULL,
    SoldQuantity INT NOT NULL CHECK (SoldQuantity > 0),
    SaleDate DATETIME NOT NULL DEFAULT GETDATE(),
    OrderId INT NULL, -- Nếu muốn liên kết trực tiếp với bảng Orders
    FOREIGN KEY (ProductId) REFERENCES Products(ProductId),
    FOREIGN KEY (OrderId) REFERENCES Orders(OrderId) -- Nếu muốn liên kết trực tiếp với bảng Orders
);
```

Cập Nhật Stored Procedure để Ghi Nhận Giao Dịch Bán

Bạn cũng nên cập nhật Stored Procedure `UpdateProductQuantityAfterSale` để nó không chỉ cập nhật số lượng sách trong bảng `Products`, mà còn thêm một bản ghi vào `SalesRecords` mỗi khi có sách được bán.

```
sql
ALTER PROCEDURE UpdateProductQuantityAfterSale
    @ProductId INT,
    @SoldQuantity INT,
    @OrderId INT = NULL -- Thêm tham số này nếu bạn muốn liên kết với đơn hàng cụ thể
AS
BEGIN
    DECLARE @CurrentQuantity INT;
    SELECT @CurrentQuantity = Quantity FROM Products WHERE ProductId = @ProductId;

    IF @CurrentQuantity >= @SoldQuantity
    BEGIN
        UPDATE Products
        SET Quantity = Quantity - @SoldQuantity
        WHERE ProductId = @ProductId;

        -- Thêm bản ghi vào SalesRecords
        INSERT INTO SalesRecords (ProductId, SoldQuantity, OrderId)
        VALUES (@ProductId, @SoldQuantity, @OrderId);
    END
    ELSE
    BEGIN
        THROW 50000, 'Số lượng sách không đủ để bán', 1;
    END
END;
GO
```

Sử Dụng trong Ứng Dụng

Khi bạn gọi Stored Procedure này từ ứng dụng C# của mình để xử lý việc bán sách, Stored Procedure sẽ tự động cập nhật số lượng sách trong bảng `Products` và ghi lại thông tin bán ra trong bảng `SalesRecords`. Điều này sẽ giúp bạn theo dõi chi tiết các giao dịch và phân tích xu hướng bán hàng.

You:

Trong thiết kế dữ liệu trên tôi nhận ra trong Product cụ thể là sách còn chứa hình ảnh. Vậy phải thiết kế lại như thế nào

---

ChatGPT:

Để bổ sung thông tin hình ảnh cho mỗi cuốn sách trong bảng `Products`, bạn có hai lựa chọn chính:

1. **Lưu trữ Đường Dẫn của Hình Ảnh**: Lưu trữ đường dẫn tới file hình ảnh trong hệ thống file hoặc trên một dịch vụ lưu trữ hình ảnh ngoại vi. Phương pháp này thường được ưa chuộng do hiệu quả và linh hoạt hơn trong việc quản lý và phục vụ hình ảnh.
2. **Lưu trữ Hình Ảnh Trực Tiếp**: Lưu trữ hình ảnh dưới dạng binary trong cơ sở dữ liệu. Cách này ít được ưa chuộng hơn vì có thể làm tăng đáng kể kích thước cơ sở dữ liệu và ảnh hưởng đến hiệu năng.

Lựa Chọn 1: Lưu Trữ Đường Dẫn Hình Ảnh

Cập nhật bảng `Products` để thêm một cột mới cho đường dẫn hình ảnh:

```
sql
ALTER TABLE Products
ADD ImageUrl NVARCHAR(MAX);
```

Trong đoạn code này, `ImageUrl` sẽ lưu trữ đường dẫn tới hình ảnh của mỗi cuốn sách. Bạn cần đảm bảo rằng ứng dụng của mình có quyền truy cập vào đường dẫn này để hiển thị hình ảnh.

Lựa Chọn 2: Lưu Trữ Hình Ảnh Trực Tiếp

Nếu bạn muốn lưu trữ hình ảnh trực tiếp trong cơ sở dữ liệu:

```
sql
ALTER TABLE Products
ADD Image VARBINARY(MAX);
```

Trong trường hợp này, `Image` là một cột kiểu `VARBINARY(MAX)` có thể chứa dữ liệu hình ảnh dưới dạng binary. Hãy lưu ý rằng việc lưu trữ hình ảnh trực tiếp trong cơ sở dữ liệu có thể gây ra vấn đề về hiệu suất, đặc biệt là khi làm việc với một lượng lớn dữ liệu hình ảnh hoặc hình ảnh có độ phân giải cao.

Khi quyết định giữa hai lựa chọn này, hãy xem xét kỹ lưỡng về yêu cầu hiệu suất, quản lý, và bảo mật của ứng dụng của bạn.

You:

Dùng lựa chọn 1. Vậy trong WPF C# có cần chỉnh sửa gì không

ChatGPT:

Khi sử dụng lựa chọn 1, tức là lưu trữ đường dẫn hình ảnh trong cơ sở dữ liệu và không lưu trữ hình ảnh trực tiếp, bạn sẽ cần thực hiện một số chỉnh sửa trong ứng dụng WPF C# của bạn để có thể hiển thị hình ảnh từ các đường dẫn này. Dưới đây là những bước bạn cần thực hiện:

1. Cập Nhật Lớp Mô Tả Sản Phẩm

Cập nhật lớp `Product` (hoặc tương đương) trong code C# của bạn để thêm một thuộc tính cho đường dẫn hình ảnh:

```
csharp
public class Product
{
    // Các thuộc tính khác...
    public string ImageUrl { get; set; }

    // Phương thức và thuộc tính khác...
}
```

2. Hiển Thị Hình Ảnh trong Giao Diện WPF

Trong file `.xaml` của bạn, bạn có thể sử dụng một `Image` control để hiển thị hình ảnh từ đường dẫn. Ví dụ, nếu bạn đang sử dụng `ListView` hoặc `DataGrid` để hiển thị sản phẩm, bạn có thể thêm một cột cho hình ảnh như sau:

```
xml
<ListView x:Name="lvProducts">
    <ListView.View>
        <GridView>
            <!-- Các cột khác -->
            <GridViewColumn Header="Image" Width="120">
                <GridViewColumn.CellTemplate>
                    <DataTemplate>
                        <Image Source="{Binding ImageUrl}" Width="100" Height="100"/>
                    </DataTemplate>
                </GridViewColumn.CellTemplate>
            </GridViewColumn>
        </GridView>
    </ListView.View>
</ListView>
```

Trong đoạn code trên, `Source="{Binding ImageUrl}"` sẽ liên kết đường dẫn hình ảnh từ mỗi đối tượng `Product` đến `Image` control trong giao diện người dùng.

3. Xử lý Đường Dẫn Hình Ảnh

Khi tải dữ liệu từ cơ sở dữ liệu, đảm bảo rằng bạn cung cấp đường dẫn hình ảnh đúng cho mỗi đối tượng `Product`. Điều này có thể bao gồm việc chuyển đổi đường dẫn tương đối thành đường dẫn tuyệt đối nếu cần.

4. Kiểm Tra Tính Hợp Lệ của Đường Dẫn

Khi làm việc với đường dẫn từ ngoài, luôn tốt khi có một số cơ chế kiểm tra để đảm bảo rằng đường dẫn hợp lệ và hình ảnh có thể được truy cập. Điều này cũng giúp bảo vệ ứng dụng của bạn khỏi các vấn đề liên quan đến đường dẫn hình ảnh không an toàn hoặc không hợp lệ.

Nếu bạn cần hiển thị hình ảnh từ một URL, hãy chắc chắn rằng ứng dụng của bạn xử lý các tình huống như hình ảnh không tìm thấy hoặc lỗi mạng một cách thích hợp.
